---

title: "Gravisonde Technical Report" subtitle: "OpenCV AI Competition 2026 — COOL Award Submission" date: "2026-10-20" authors: ["Gravisonde Team"] tags: ["opencv", "aws", "graviton", "cool", "computer-vision", "benchmark"]

Gravisonde Technical Report

OpenCV AI Competition 2026 — COOL Award Submission

---

1. Problem & Target Users

Computer-vision workloads are CPU-bound and expensive on x86 instances. AWS Graviton (Arm) promises 20–40% better price-performance, but most teams still run stock OpenCV on x86 because the migration path and evidence are opaque. Gravisonde provides a reproducible, live, end-to-end pipeline that proves the gain on real video with OpenCV 5 and the Cloud-Optimized OpenCV Library (COOL).

Target users: Hackathon judges (primary), OpenCV/AWS engineers, edge/ops teams evaluating Graviton migration for CV workloads.

---

2. Proposed Solution

A live video analytics service running OpenCV 5 object detection on an AWS Graviton4 instance (`c8g.xlarge`) using the COOL AMI. The pipeline captures frames, runs inference via OpenCV's new DNN engine (with ONNX Runtime backend), and emits per-frame metrics to CloudWatch. A Next.js 16 dashboard streams these metrics in real time via Server-Sent Events and displays live FPS, latency percentiles, and cost-efficiency (fps/$).

---

3. What Makes It Meaningfully Different

- Directly targets the **COOL Award** ($1,000) by proving COOL on Graviton beats stock x86.

- Uses **OpenCV 5** — new DNN engine (64% ONNX spec), ALIKED features, LightGlue matcher.
- Live, measurable demo — judges see real numbers updating in **under 5 seconds**.
- Reproducible: one-command deploy (`make deploy-infra`), benchmark CSV download, public repo with `docker-compose.yml` for local dev.

---

4. Why OpenCV 5 + AWS Are Necessary

- **OpenCV 5:** The new DNN engine with ORT integration is required to run modern ONNX models (YOLO11n) efficiently. ALIKED/LightGlue showcase the new Features2D module.
- **AWS Graviton + COOL:** COOL is a Graviton-optimized OpenCV 5 build (Arm KleidiCV). It's only available as an AWS Marketplace AMI — making AWS mandatory, not optional.

---

5. Core User Journey

1. Judge opens `https://gravisonde.demo`
2. Sees live video with bounding boxes updating at 25+ FPS
3. Reads top-bar: `c8g.xlarge • COOL 5.0.0 • 28.4 fps • $0.136/hr • 209 fps/$`
4. Clicks "Download Benchmark" → gets CSV with 5,000+ frames of raw timing data
5. Opens GitHub repo → reads README → sees `make up` reproduces it locally

---

6. Minimum Viable Product (MVP)

Feature	Description	Judging Criterion
Video ingestion	RTSP/file → ffmpeg → OpenCV VideoCapture	Substantive OpenCV 5 usage
Detection pipeline	YOLO11n (DNN) + ALIKED/LightGlue (Features2D)	OpenCV 5 new features
COOL runtime	Pipeline runs on COOL AMI on c8g.xlarge	COOL Award requirement
Metrics collection	Per-frame timing → CloudWatch + ring buffer	Evidence & technical quality
Next.js 16 dashboard	App Router, Cache Components, Tailwind v4	Working demo + modern stack
Live metrics UI	Real-time charts via SSE	Demo quality
Architecture diagram	Mermaid diagram in repo	Required deliverable
Technical report	Markdown with responsible-use section	Required deliverable
Demo video (≤5 min)	Scripted walkthrough	Required deliverable

---

7. Stretch Features

- Side-by-side comparison: toggle COOL vs Stock in same dashboard
- Kinesis Video Streams for multi-camera ingestion
- Multi-region failover with CloudWatch cross-region dashboards
- Automated nightly benchmark CI on Graviton self-hosted runner

---

8. Technical Architecture

See Architecture Diagram (architecture.mmd) for the interactive diagram.

Summary: Camera → EC2 Graviton (COOL AMI) → CloudWatch/S3 → Next.js 16 (Vercel) → Judge's browser.

---

9. Evaluation Evidence & Limitations

Evidence Collected

- Per-frame CSV: timestamp, latency_ms, inference_ms, detections, instance_id, versions
- CloudWatch custom metrics: FPS, p50/p95/p99 latency, detection rate
- Benchmark report: COOL vs Stock comparison on identical hardware (AMI swap)

Known Limitations

- Single-instance variance — no statistical significance across fleet
- Video source bias — results specific to sample content (Big Buck Bunny / CC0 footage)
- COOL AMI version pinning — gains may differ with future COOL releases
- Model quantization effects not evaluated (FP32 vs FP16 vs INT8)
- Network latency not included (local file source used)

---

10. Responsible Use

No PII collected; video is processed locally on EC2, never uploaded. The demo uses only CC0/public-domain sample videos. No facial recognition, biometric analysis, or surveillance capabilities are implemented. The system is designed for performance benchmarking, not production deployment.

Appendix: Reproduction Commands

```
# Local development
make local-up          # Starts dashboard + pipeline + LocalStack at localhost:3000

# AWS deployment
make bootstrap         # CDK bootstrap (once per account/region)
make deploy-infra      # Provisions EC2, S3, CloudWatch, IAM
make build-pipeline    # Builds COOL + Stock Docker images
make start-pipeline    # Starts pipeline on EC2 via SSM
make benchmark         # Runs COOL vs Stock comparison, uploads CSV to S3
```

Repository Structure

```
gravisonde/
├── apps/
│   ├── dashboard/          # Next.js 16 dashboard (App Router, React 19, Tailwind v4)
│   └── cv-pipeline/        # OpenCV 5 Python pipeline (DNN + Features2D)
├── infra/                  # AWS CDK (TypeScript) — VPC, EC2, IAM, S3, CloudWatch
├── scripts/                # bench_runner.py, stub_api.py
├── docs/
│   ├── technical-report.mdx    # This file
│   ├── architecture.mmd        # Mermaid architecture diagram
│   ├── reproduction-guide.md   # Step-by-step reproduction guide
│   └── demo-script.md          # Demo walkthrough script
├── docker-compose.yml      # Local dev stack
├── Makefile                # All automation targets
└── DECISIONS.md            # Build decisions log
```

License

MIT License — see LICENSE file for details.

Generated for the OpenCV AI Competition 2026 COOL Award.